Code Plagiarism Checker
User Guide

SDN260S Software Design II · Group 8 · Topic 17

1. About this guide

This guide is for anyone who wants to use the Code Plagiarism Checker to compare Java submissions, for example a lecturer or tutor marking a set of assignments. You do not need to read the source code. If you want to build the program, run the unit tests or see how it is put together, the README in the project folder covers that.

The checker reads a set of .java files, compares every file with every other file, and ranks the pairs from most to least similar. It marks the pairs that score high enough to deserve a closer look, and it shows which parts of the code match.

One thing to keep in mind throughout. The checker measures similarity. It does not decide whether anyone plagiarised. Two students working from the same assignment can honestly write similar code, especially in short programs. A flag only means that a person should read the two files side by side.

2. Before you start

What you need

- **Java 11 or newer** (a JDK, not only a JRE). Check with java -version.
- The project folder, already compiled (the README shows how to compile it). The compiled classes live in target/classes.
- The .java files you want to compare, saved in one folder if possible.

How you start it

Open a terminal in the project folder. The program has three ways in. Pick whichever suits you. They all use the same comparison, so the scores are the same.

You want to	Type this (Linux or macOS)	On Windows
Use the window	java -cp target/classes plagiarismchecker.Main --gui	use target\classes
Use the text menu	java -cp target/classes plagiarismchecker.Main --cli	use target\classes
Compare in one command	java -cp target/classes plagiarismchecker.Main <file or folder> ...	use target\classes

Run the command from the project folder. The program writes its log to a logs folder in the place you ran it from.

If you would rather let Maven start the program, compile the project first with mvn compile, then this works too (the first run needs internet to download a plugin). Change --cli to --gui, or to a file or folder path, for the other modes. The command is split over two lines here to fit the page; type it as one line:

```
mvn org.codehaus.mojo:exec-maven-plugin:3.1.0:java
    "-Dexec.mainClass=plagiarismchecker.Main" "-Dexec.args=--cli"
```

3. Quick start: five minutes with the sample files

The project comes with eleven invented submissions in samples/mock. They are not real students' work. Some pairs were built to look alike on purpose, so they are a safe way to see what the checker does.

1. Open a terminal in the project folder.
2. Type: java -cp target/classes plagiarismchecker.Main samples/mock
3. Wait a second or two. The program lists the eleven files it loaded, then prints the ranked pairs.
4. Look at the top of the list. Three pairs are marked “flagged for review”: Alice and Bob (factorial), Gina and Hank (bank account), and Dan and Eve (palindrome).
5. Now try the window with the same files (section 4) and click through the pairs.

The folder test_datasets has nine more small sets, one per test case, with a README that says what to expect from each. They are good for trying empty files, wrong file types and large files.

In that run, 11 files gave 55 pairs and 3 were flagged. Below those three, the next pair (Gina and Kim) scored 61%. It is a partial copy that sits under the flag line, which is a good reminder that the threshold is a rule of thumb, not a wall.

4. Using the window

Start it with --gui. A window titled “Code Plagiarism Checker - SDN260S Group 8” opens.

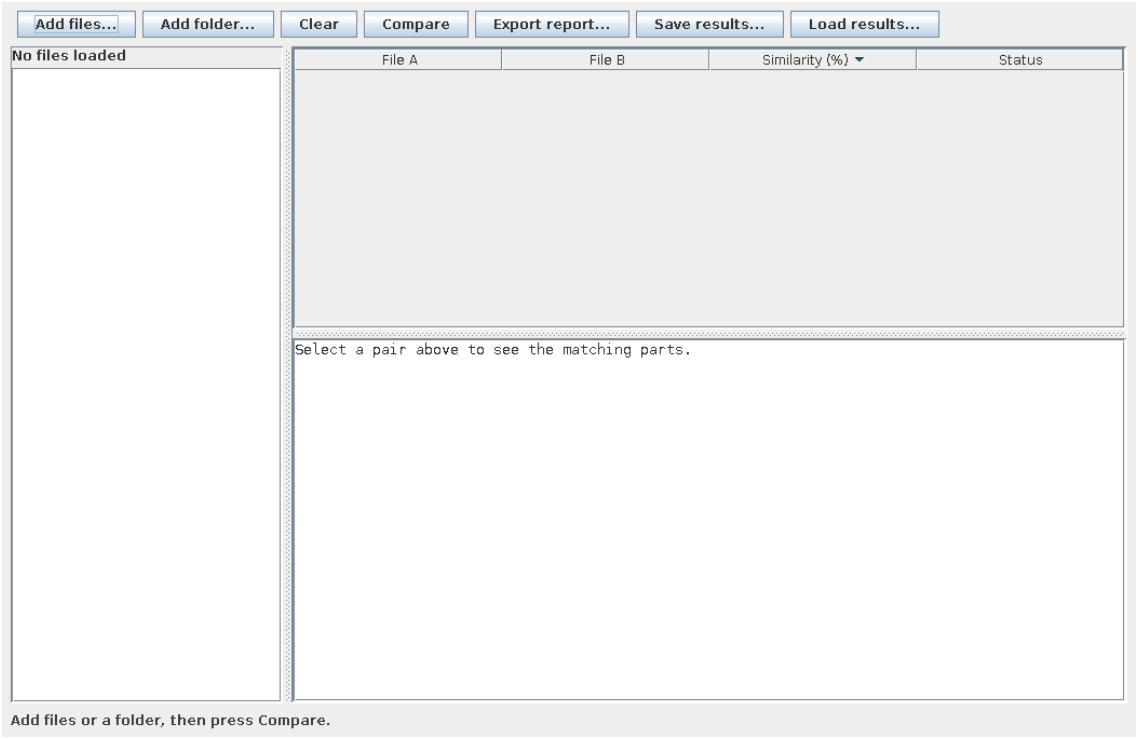


Figure 1: The window when it first opens

Along the top is a row of buttons. The left panel will list your files. The table on the right will list the pairs, and the box underneath will show the matching parts of whichever pair you click. The line at the very bottom tells you what the program is doing.

Step by step

1. Press **Add folder...** and choose the folder with the submissions. Every .java file inside it and inside its sub-folders is loaded. Or press **Add files...** to pick individual files. You can add more than once. A file added twice is only counted once.
2. Check the left panel. It shows how many files were loaded and each file’s name and line count. Anything that could not be loaded (an empty file, a file that is not .java, and so on) is left out and the reason goes in the log.
3. Press **Compare**. You need at least two files. The buttons switch off while the program works, and then the table fills in.
4. Read the table. Each row is one pair, with the two file names, the similarity as a percentage, and a status of “Flagged for review” or “Not flagged”. The most similar pairs are at the top, and flagged rows are shaded. Click a column heading to sort by it.
5. Click a row. The box below the table shows the matching parts of the code for that pair (section 6 explains how to read them).
6. When you are done, press **Export report...** for a text or CSV report, or **Save results...** to keep the results in a .g8 file you can open later with **Load results...**. **Clear** empties the lists so you can start on a different set.

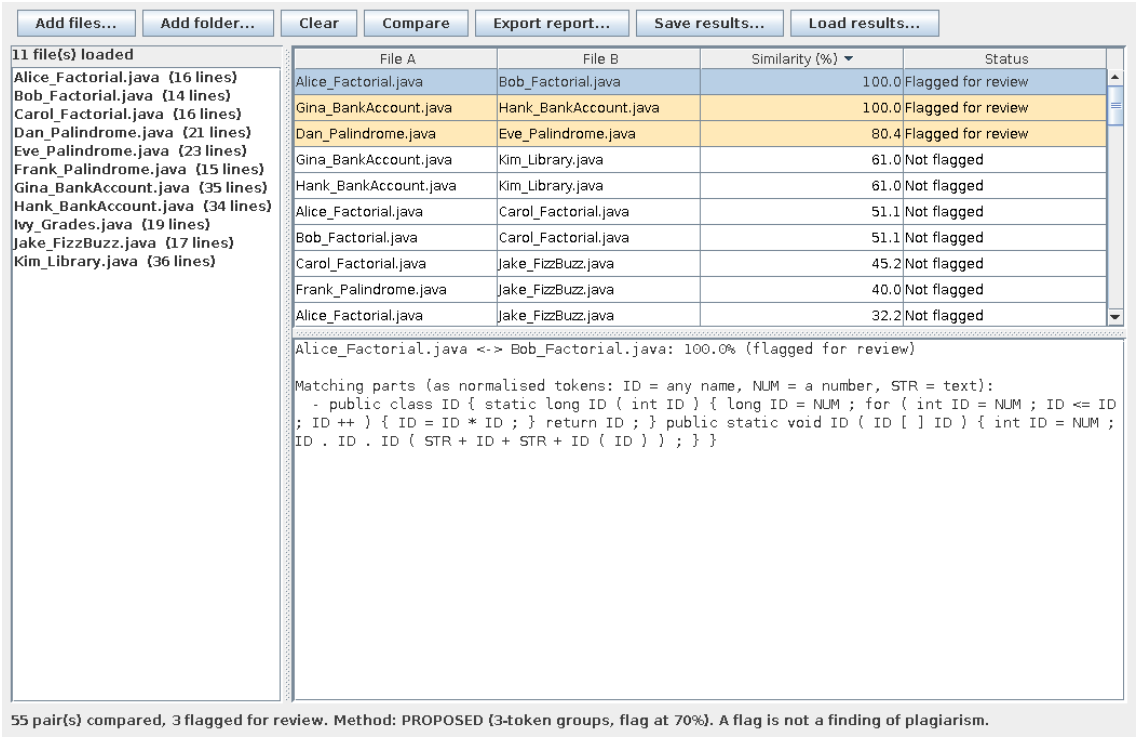


Figure 2: Eleven sample files compared. The first pair is selected

Button	What it does
Add files...	Choose one or more .java files
Add folder...	Choose a folder; sub-folders are searched
Clear	Empty the file list and the results
Compare	Compare every pair (needs two or more files)
Export report...	Write the results to a .txt or .csv file
Save results...	Save the results as a .g8 file
Load results...	Open a .g8 file saved earlier

If you press Compare with fewer than two files, or Export or Save before comparing, the window politely tells you what is missing instead of doing anything.

5. Using the text menu

Start it with --cli. The menu appears and comes back after every action.

```
--- Code Plagiarism Checker ---
1. Add submissions (file or folder)
2. View submissions
3. Compare
4. View results
5. Generate report
6. Exit
Choose:
```

1. Type **1**, press Enter, then type the path to a .java file or to a folder. The program replies with how many files are loaded in total. Repeat to add more.
2. Type **2** to see the numbered list of what is loaded.
3. Type **3** to compare. It tells you how many pairs it compared.
4. Type **4** to see the ranked results.
5. Type **5** to write a report. Give a name ending in .txt or .csv, or just press Enter to use plagiarism-report.txt.
6. Type **6** to leave.

If a file does not load, the menu only says how many files are loaded in total. The reason (the file is empty, it is not a .java file, the path is wrong) is written to the log file, so look in logs/plagiarism-checker.log. Also type paths without quotation marks. A path that contains a character your system does not allow can stop the menu with an error, and on Windows quotation marks count as one of those characters.

If you type something that is not a number from 1 to 6, the menu says “Please choose a number from 1 to 6.” and shows itself again. Choosing Compare or View results too early gives a short message telling you what to do first. The menu does not save or load .g8 files; that is done in the window.

6. Understanding the results

The score

Before comparing, the checker removes comments, import and package lines and extra spacing. It then turns the code into tokens, where every name you chose (variables, methods, classes) becomes the same placeholder, ID. It cuts the tokens into overlapping groups of three and works out how many groups the two files share compared with how many groups they have between them. Shared groups divided by all groups, times 100, is the similarity.

Because names are ignored, renaming variables and rewriting comments does not hide copied logic. Because it looks at groups of tokens, moving one statement around only changes a few groups.

What the numbers looked like on the sample files

Kind of pair (sample files)	Score
Renamed copy (Alice and Bob, Gina and Hank)	100.0%
Renamed and reordered copy (Dan and Eve)	80.4%

Kind of pair (sample files)	Score
Partial copy (Gina and Kim)	61.0%
Independent solutions to the same task	about 29% to 51%
Unrelated programs	about 16% to 45%

The flag line is at 70%. Scores in the 40s and 50s are very common between honest programs, because short programs share ordinary structure such as loops and print statements. A middling score tells you very little by itself.

Reading the matching parts

The matching parts are shown as normalised tokens. The legend is: **ID** is any name, **NUM** is any number, **STR** is any text in quotes, and symbols such as braces and brackets appear as they are written. Here is the match for the sample pair StudentA and StudentB, which do the same job with different names and spacing:

```
public class ID { public static void ID ( ID [ ] ID ) { int ID = NUM ;
for ( int ID = NUM ; ID < NUM ; ID ++ ) { ID += ID ; } ID . ID . ID ( ID ) ; } }
```

That is the whole of both programs, which is why the score is 100%. When a pair matches only in small pieces, you will see several short fragments separated in the list instead of one long one. Long matching runs that go beyond ordinary boilerplate are what deserve your attention.

What to do with a flagged pair

- Open both original files and read them side by side. The tool narrows the search; the judgement is yours.
- Think about the assignment. If it only allows one sensible solution, high similarity is expected.
- Look for unusual things the two files share that were not required, such as the same odd variable names in comments, the same unusual structure, or the same mistakes.
- Follow your institution’s own process before saying anything to a student. This tool is not evidence on its own.

7. Reports, saved results and the log

Text report

A text report has a header with the date, the number of pairs and how many were flagged, then every pair in order with up to ten matching fragments each. This is a real one for the three small sample files:

```
CODE PLAGIARISM CHECKER - REPORT
Generated: 2026-10-31 17:45:54
A flag only marks a pair for a person to review. It is not a finding of
plagiarism.
Pairs compared: 3, flagged for review: 1

1. StudentA.java <-> StudentB.java: 100.0% (flagged for review)
     matching tokens: public class ID { public static void ID ( ID [ ] ID )
{ ...
2. StudentA.java <-> StudentC.java: 27.8%
     matching tokens: public class ID { public static void ID ( ID [ ] ID ) {
     matching tokens: ) { ID
...
```

CSV report

Give the file a name ending in .csv and you get one row per pair that opens in Excel, with the columns File A, File B, Similarity (%), Flagged for review (Yes or No) and Matching fragments.

Saved results

In the window, **Save results...** writes a .g8 file. **Load results...** brings it back, so you can look at an old comparison without running it again. The program only opens its own files. If you try to open some other file, you get an error message and nothing else happens.

The log

Every run adds lines to logs/plagiarism-checker.log. Each line has the time, a level (INFO, WARN or ERROR) and a message. It records every file loaded, every file skipped and why, and every comparison. In command-line mode the same lines also appear on screen. In the text menu and the window they appear only here, so if something seems to be missing from your results, look here first.

8. Messages you may see

When you run the program with file or folder paths on the command line, these messages show on screen. In the text menu and in the window you will only see a short summary such as “0 file(s) loaded in total” or “No usable .java files were found”, and the full message is in the log.

Message	**What it means and what to do**
File does not exist: …	The path is wrong. Check the spelling, and put quotes around paths with spaces.
Only .java files can be checked: …	That file was skipped. Rename it or leave it out.

Message	What it means and what to do
File is empty: …	The file has no text, or only blank space. It was skipped.
Skipped symbolic link	A shortcut inside a folder was ignored on purpose.
… is not valid UTF-8; read as ISO-8859-1	The file was saved by an older editor. It was still read, so no action is needed.
No usable source files were loaded	Nothing could be read from what you gave it. Check the log for the reasons.
Add at least two files to compare them	Only one file was loaded. Add another.
Please choose a number from 1 to 6	In the text menu, type a number from the list.

9. Good to know

- Only .java files are checked.
- When you give the program a whole folder, files that do not end in .java are skipped without a message. Files you list one by one are rejected with a message.
- A file that holds nothing but comments and imports loads, but it has nothing to compare, so it scores 0.0% against everything.
- The group size (3) and the flag threshold (70%) are fixed for now. They are the values the group approved on 1 November 2026.
- The checker compares the files you give it. It does not look anything up on the internet or in earlier years' work.
- It looks at how the code is built, not what it does. Two programs that behave the same but are written very differently can score low.
- With a large set, comparing takes longer because the number of pairs grows quickly: 100 files make 4,950 pairs. On a computer with several processor cores, the pairs are compared in parallel.
- Please only use real student work in line with your institution's rules on privacy and academic integrity. The sample files are invented.

10. Troubleshooting

Problem	Try this
"Could not find or load main class"	The program has not been compiled, or you are in the wrong folder. Compile it (see the README) and run from the folder that contains target.
The window does not appear	--gui must be the only thing after the class name. A remote or screen-less computer cannot show a window.
The menu stopped with a long Java error after I typed a path	Start it again and type the path without quotation marks or unusual characters.
A file you added is missing from the list	Check the log file for the reason. In the text menu and the window it is not shown on screen.

Problem	Try this
The list of pairs is empty	Fewer than two usable files were loaded. Look at the file list and the log.
A pair you expected is missing	One of its files was probably skipped. The log says why.
The numbers look odd	Remember that comments, imports and names are ignored. Look at the matching parts.